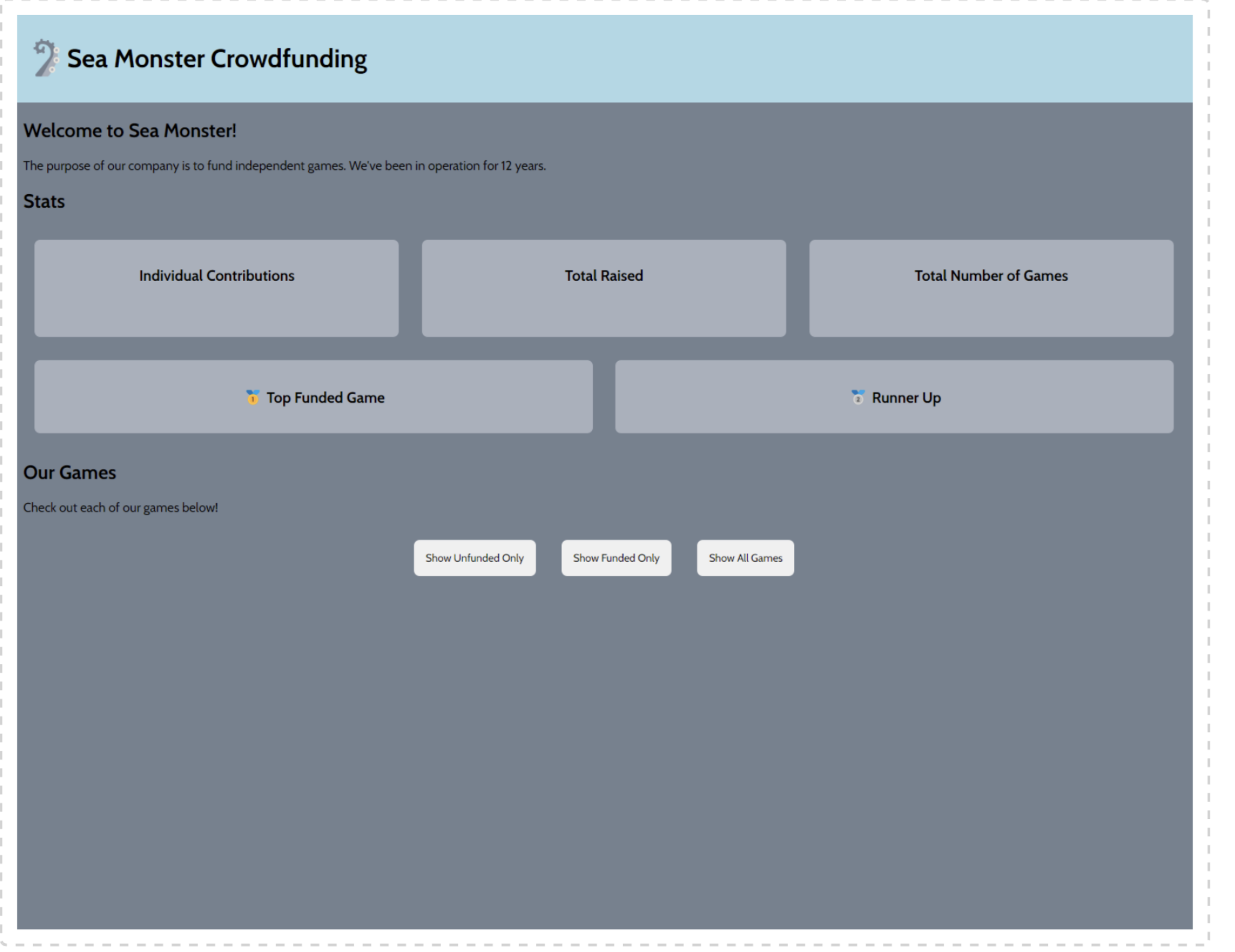
Challenge 3 of 7: Add data about each game as a card to the games-container

✅ Checkpoint

At this point, after changing some of the CSS, your website should look something like the screenshot below. Compare your work with the exemplar and see if there are any changes you might need to make!

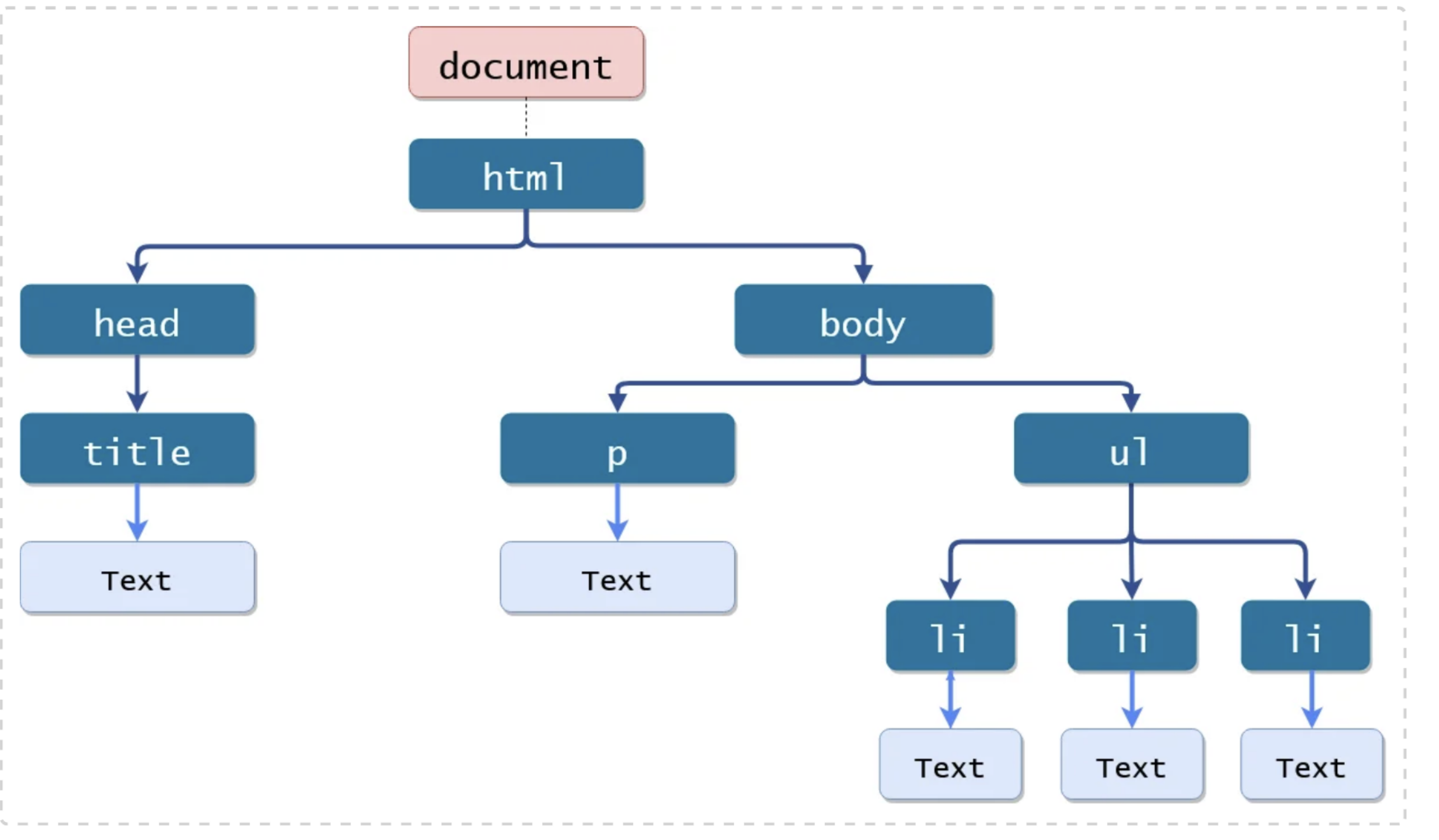


📝 Background

To solve this challenge, you will need to use several different skills. Review each section below before beginning the challenge. If you feel comfortable with a topic, skim the corresponding section as a refresher first.

The DOM

You'll notice that line 26 of **`index.js`** in the starter code grabs an element from the DOM and saves it to a JavaScript variable, allowing us to add or remove elements to and from the **`games-container`**. Recall that the DOM is a model that defines a tree structure for HTML elements, with an object at each node in the tree representing a part of the HTML document. Here's a simple illustration of the DOM (image credit: Tutorials Tonight):



If you need a refresher on working with the DOM, here are some great resources:

- JavaScript DOM - Getting Started
- JavaScript HTML DOM

Some DOM-related stuff you may need in this challenge:

- **`document.createElement(/* some HTML tag here */)`** - creates an HTML element
- **`element.innerHTML = /* HTML goes here */`** - sets the inner HTML of an element
- **`element.classList.add(/* string with class name here */)`** - adds a class to the given element

Types of Functions in JavaScript

Throughout the upcoming challenges, you'll be working with functions. Functions group code statements together. In order to be used, functions must be both defined and called. There are two different types of functions in JavaScript, distinguished by how they are defined: arrow functions and traditional functions declared using the **`function`** keyword. Here's an example of equivalent functions written in each format:

```
// arrow function saved to a variable
const addChinchilla = (newChinchilla) => {
  chinchillas.append(newChinchilla);
  alert("New chinchilla added to chinchilla farm!");
}

// traditional function declared using the function keyword
function addChinchilla(newChinchilla) {
  chinchillas.append(newChinchilla);
  alert("New chinchilla added to chinchilla farm!");
}
```

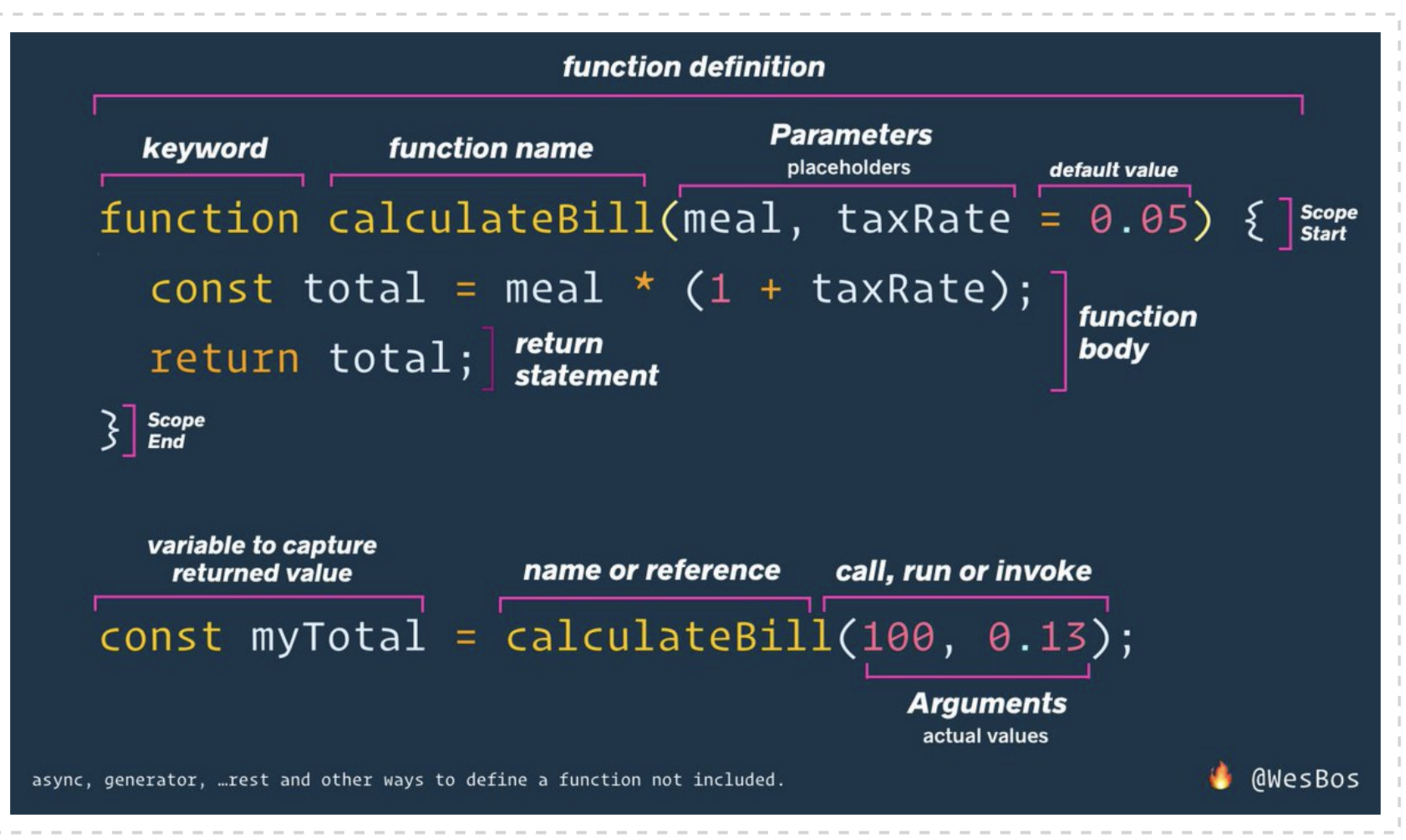
For the most part, these two functions will behave identically. However, there are some technicalities you will run into that differentiate the two:

- **The *`this`* keyword** behaves differently, which is the primary difference between the two
- **Arrow functions cannot be used as a constructor** with the **`new`** keyword
- **Arrow functions do not have an arguments object**, whereas traditional functions do
- **Arrow functions have implicit return** without the **`return`** keyword, whereas traditional functions will return **`undefined`** if there is no **`return`** keyword

If you'd like to read more about how the **`this`** keyword behaves and specific reasons why you might choose one function type over the other, check out this resource:

- Regular vs Arrow Functions

Check out this detailed diagram explaining how to define a traditional function as well as how to call it (image credit: Wes Bos):



As you can see in the diagram above, functions have their own **scope**, which refers to the section of code within which a variable is valid. If a variable is declared using **`let`** or **`const`** within the curly braces of a function, its scope is said to be that function. The variable will be unavailable outside the function.

```
// the scope of this variable is the entire script
const narwhals = getNarwhals();

const calculateMigrationDistance = () => {
  // the scope of this variable is just this function
  let total = 0;

  for (let i = 0; i < narwhals.length; i++) {
    total += narwhals[i].distanceMigrated;
  }
}

// at this point in the code, the variable total is no longer accessible
```

You will see both arrow functions and traditional functions very frequently when working with React in this course. It's highly useful to be familiar with declaring and calling both types!

Template Literals

Template literals allow you to execute JavaScript code within a string. They're useful when you want to use variables, make quick calculations, or execute functions within a string.

To use template literals, wrap the string within backticks (`` ` ``), the key just above the tab key. Within the string, use **`${}`** to indicate sections of JavaScript code which should be executed. Here are some examples:

```
const greeting = `Hi ${ name }, how are you doing?`;
// => Hi Harvey, how are you doing?

const queueMsg = `You are currently #${ getQueuePosition(id) } in the queue.`;
// => You are currently #17 in the queue.

const bigString = `Template literals can also spill over multiple lines.
                   We have used ${(greeting.length + queueMsg.length) / 2}
                   characters on average in our strings so far.`;
// => Template literals can also spill over multiple lines. We have used 45
characters
// => on average in our strings so far.
```

Here's an example of how template literals can be used in web development (image credit: DEV community):

🥏 Challenge Steps

All that said, let's tackle this challenge! As a reminder, here is some DOM-related stuff you may need in this challenge:

- **`document.createElement(/* some HTML tag here */)`** - creates an HTML element
- **`element.innerHTML = /* HTML goes here */`** - sets the inner HTML of an element
- **`element.classList.add(/* string with class name here */)`** - adds a class to the given element

➡️➡️➡️ Step 1

Create a **`for`** loop within the **`addGamesToPage`** function that loops over each item in the argument **`games`**. You can expect that **`games`** will be an array of objects with the same structure as **`GAMES_JSON`**.

🔑 **Secret Key component 1**: How many times will this loop run when it is called with the argument **`GAMES_JSON`**?

➡️➡️➡️ Step 2

Within the loop you created, create a new **`div`** element which will become the game card that displays info about a game. Add the class **`game-card`** to the **`div`**'s class list.

➡️➡️➡️ Step 3

Use a template literal to set the inner HTML of the **`div`** you created so that it displays some information about the game. **Include the game's image and at least 2 of the game's other attributes.** Remember that inner HTML can contain HTML elements.

- Make sure to give the image the class **game-img**.

🔑 **Secret Key component 2**: What would the output of the following code segment be?

```
let x = 10;
let y = 4
console.log("The answer to ${x} + ${y} is ${x + y}");
```

1. **`The answer to $10 + $4 is $14`** (keyword: orange)
2. **`The answer to ${x} + ${y} is ${x + y}`** (keyword: seafoam)
3. **`The answer to 10 + 4 is 14`** (keyword: yellow)
4. **`The answer to 14 is 14`** (keyword: lilac)

➡️➡️➡️ Step 4

Append the newly created **`div`** to the correct element in the DOM so that the game cards appear under the "Our Games" heading.

➡️➡️➡️ Step 5

After declaring the **`addGamesToPage`** function, call the function with the correct variable so that all games are added to the page.

🔑 **Secret Key component 3**: Which variable was passed to **`addGamesToPage`** in order to add all the games to the page?

🚀 Moving on to Challenge 4

Create the 🔑 Secret Key from its components with no spaces in between:

- Component 1: How many times will this loop run when it is called with the argument **`GAMES_JSON`**?
- Component 2: What would the output of the following code segment be?
- Component 3: Which variable was passed to **`addGamesToPage`** in order to add all the games to the page?

Hint: your 🔑 **Secret Key** should be 19 characters long.

Reminders about 🔑 **Secret Keys**:

- Keys for the Offline PDF version of prework **are case sensitive**!
  - (For example, "SEA_MONSTER" is NOT the same as "sea_monster")
- Secret keys can be any combination of letters, numbers, and symbols.

Use your 🔑 **Secret Key** to unlock the next PDF file. If your 🔑 **Secret Key** is correct, you will move onto the next step!